

SDC 학생강사

컴퓨터공학 입문자를 위한 자료구조론

2주차

CONTENTS

01

ADT

해당 목차에 대한 설명을 입력해 주세요

02

스택

해당 목차에 대한 설명을 입력해 주세요

03

큐

해당 목차에 대한 설명을 입력해 주세요

01

ADT

ADT

자료구조에 대한 설명, class로 구체화 하기 전 기능을 정리해놓은 것

추상 자료형의 내용 :

- 저장할 데이터에 대한 설명 (무엇을 저장하나?)
 - 저장하는 데이터에 대해 어떤 기능을 할 수 있나?
 - 오류에 대한 처리 방법
- | | |
|--|-----------|
| | class |
| | => 멤버 변수 |
| | => 멤버 함수 |
| | => 오류 핸들러 |

Example: ADT modeling a simple stock trading system

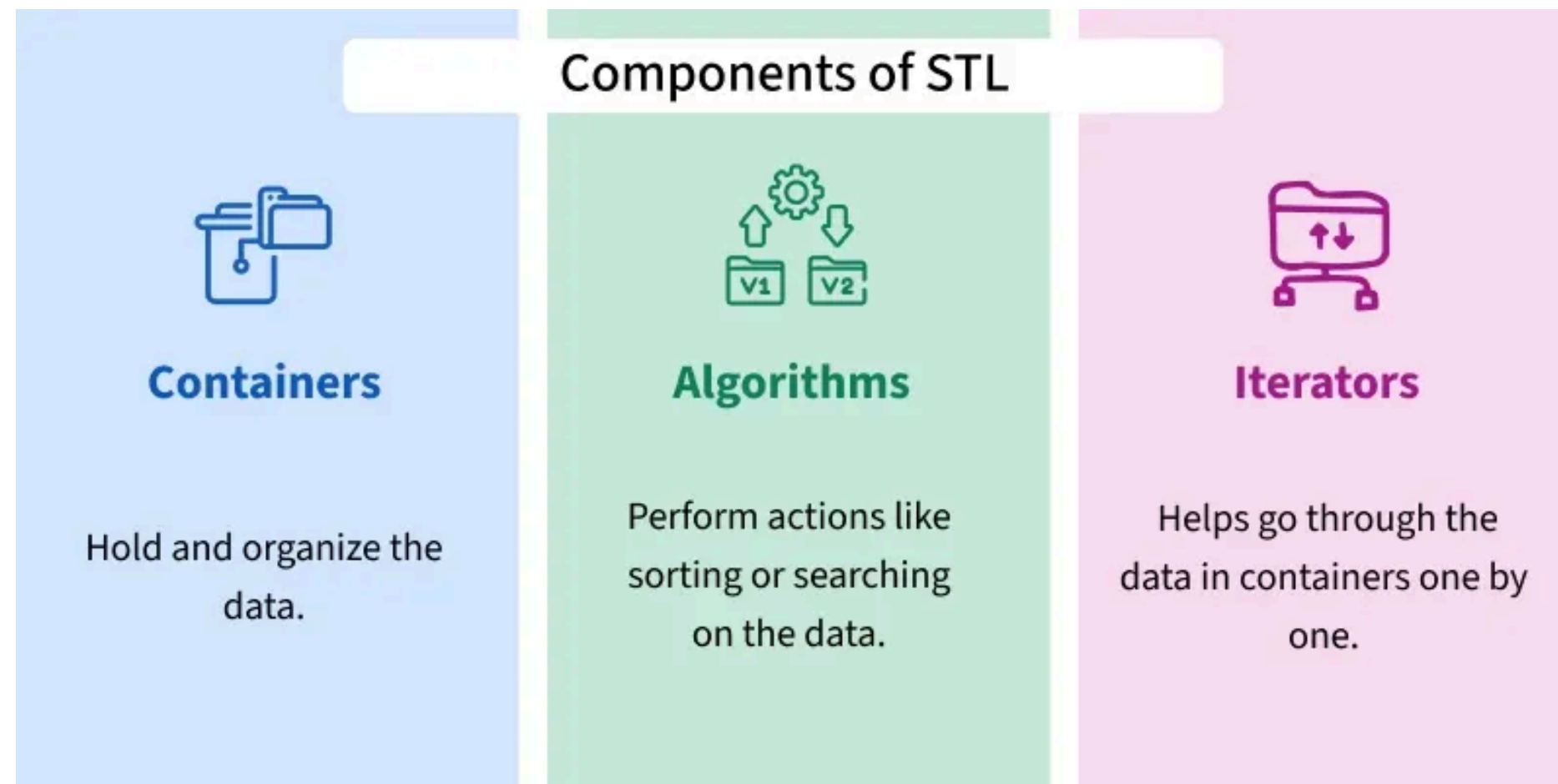
- The data stored are buy/sell orders
- The operations supported are
 - ♦ order **buy**(stock, shares, price)
 - ♦ order **sell**(stock, shares, price)
 - ♦ void **cancel**(order)
- Error conditions:
 - ♦ Buy/sell a nonexistent stock
 - ♦ Cancel a nonexistent order

01

ADT

STL Standard Template Library

여러 자료구조, 함수, 알고리즘을 쓰기 쉽게 정형화해 라이브러리화 해놓은 것



STL 컨테이너

1. Sequence Containers : Vector, Deque, List, Forward List, Array.
2. Container Adaptors : Stack, Queue, Priority Queue
3. Associative Containers : Set, Multiset, Map, Multimap
4. Unordered Associated Containers : Unordered Set, Unordered Multiset, Unordered Map, Unordered Multimap

우리 수업에서 배우고자하는건 STL 사용법이 아니라 STL이 어떻게 구성되어 있고, 어떤 원리로 작동하며, 언제 어떻게 사용할지, 자료구조에 대한것을 배우는것이 목적, STL은 사용하지 않음,

STL은 문제해결기법에서 쓸거니 공부하긴 해야함

02

스택

Stack ADT stack 자료구조에 필요한 기능을 먼저 명세해놓음

push - 요소를 삽입

pop - 요소를 제거함

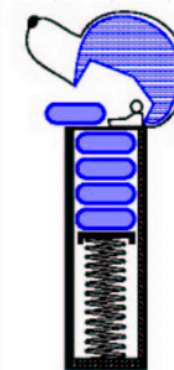
top - stack의 가장 위에 있는 요소를 반환하나 제거하지 않음

size - stack에 있는 요소의 수를 반환함

empty - stack이 비어있는지 아닌지 반환함

The Stack ADT

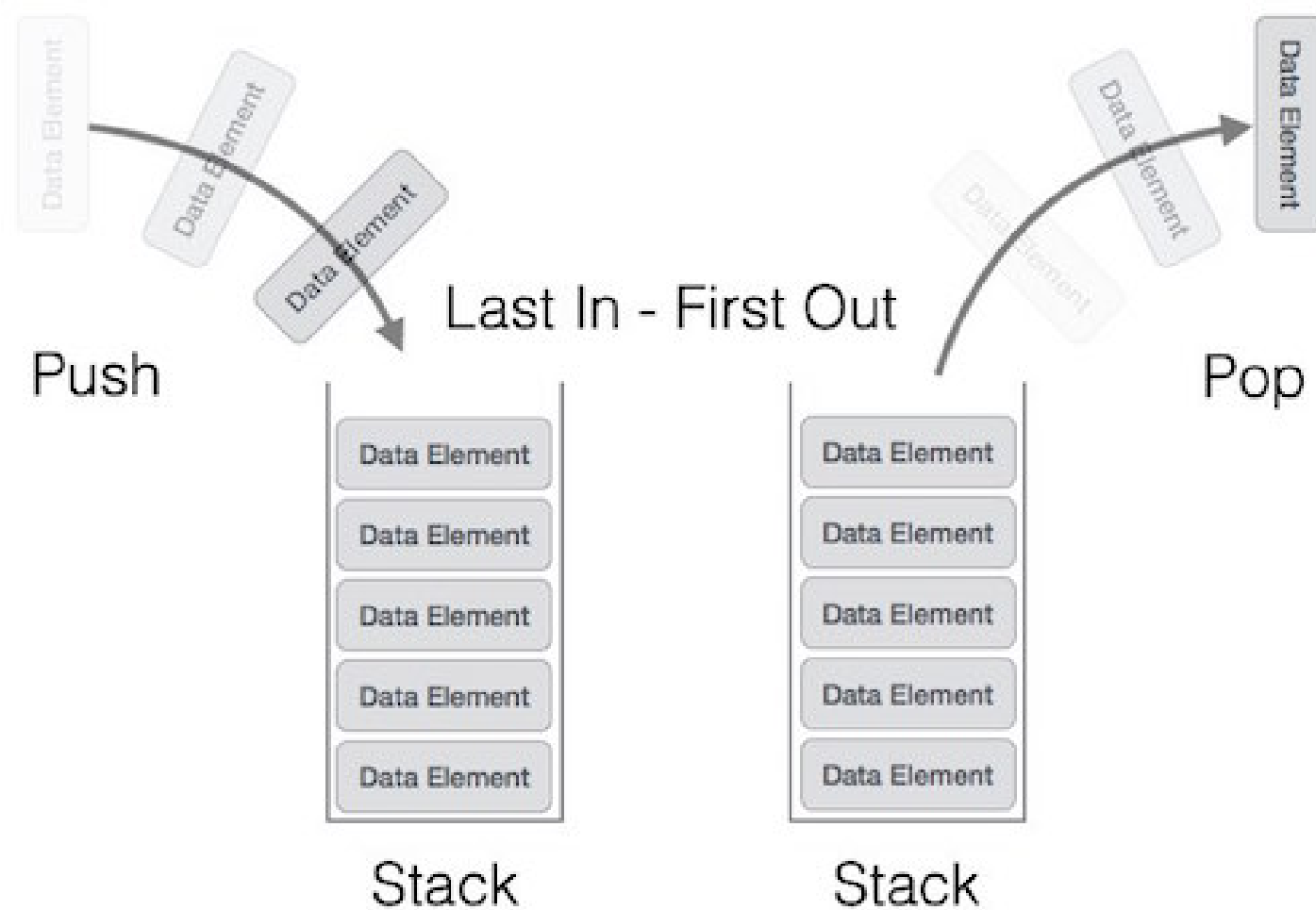
- The **Stack** ADT stores arbitrary objects
- Insertions and deletions follow the last-in first-out scheme
- Think of a spring-loaded plate dispenser
- Main stack operations:
 - **push**(object): inserts an element
 - **object pop**(): removes the last inserted element
- Auxiliary stack operations:
 - **object top**(): returns the last inserted element without removing it
 - **integer size**(): returns the number of elements stored
 - **boolean empty**(): indicates whether no elements are stored



02

스택

Stack



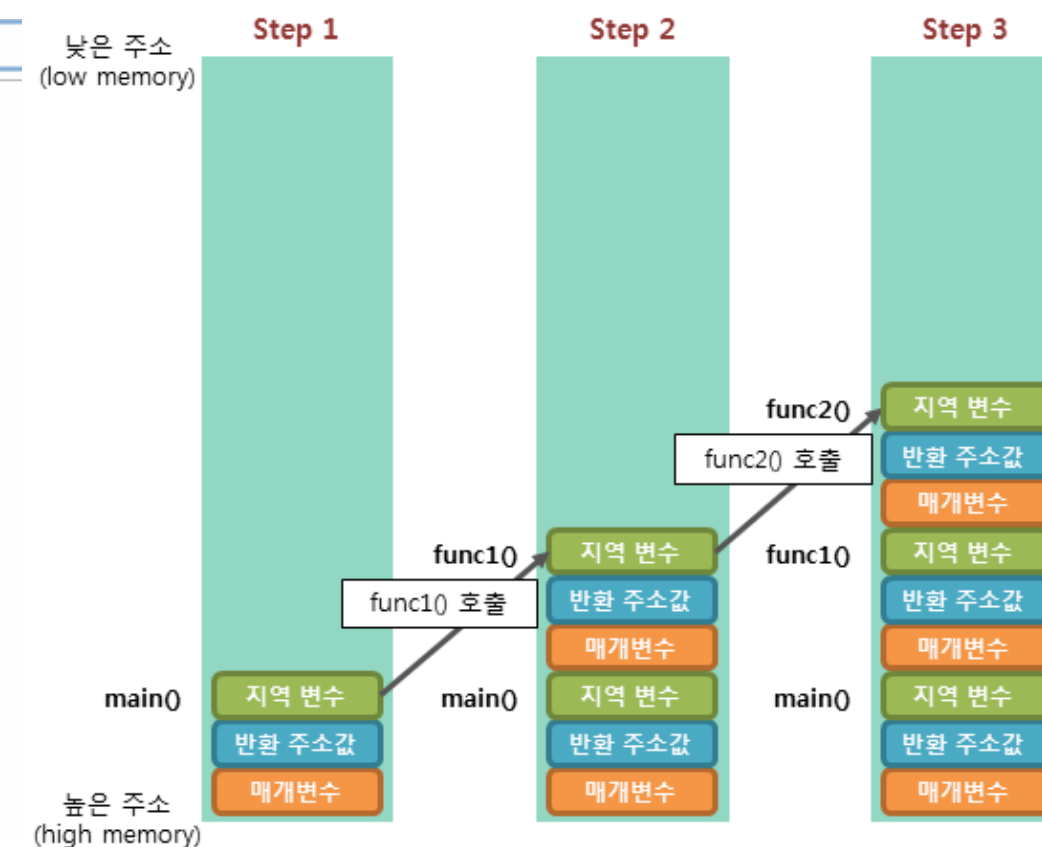
← → ↻ [안전함 https://pushstate-idlfegooqk.now.sh/ajax](https://pushstate-idlfegooqk.now.sh/ajax)

Ajax 뒤로 가기 예제

게시글 목록

- [제목 1](#)
- [제목 2](#)
- [제목 3](#)
- [제목 4](#)
- [제목 5](#)
- [제목 6](#)
- [제목 7](#)
- [제목 8](#)
- [제목 9](#)
- [제목 10](#)

[다음](#)



Stack Interface

Stack Interface in C++

- C++ interface corresponding to our Stack ADT
- Uses an exception class `StackEmpty`
- Different from the built-in C++ STL class `stack`

```
template <typename E>
class Stack {
public:
    int size() const;
    bool empty() const;
    const E& top() const
        throw(StackEmpty);
    void push(const E& e);
    void pop() throw(StackEmpty);
}
```

ADT를 기반으로 실제 stack을 class로 구현함

stack은 하나의 동일한 데이터 타입을 저장 할 수 있으며,
데이터 타입의 종류는 상관이 없다<typename E>

예외처리 throw(StackEmpty) - 비어있는 stack에 대해
pop, top 연산을 하려고 할때 오류 발생

02

스택

배열을 통한 구현

데이터를 배열을 이용해 저장하고, 배열에 대한 함수를 class로 관리한다.

왼쪽에서 오른쪽으로 (0번 인덱스 → top)만 데이터를 쌓아간다

배열 자체는 index를 통해 접근 할 수 있지만
stack을 구현하기때문에 top으로만 접근하게 제한해야한다.

```
1  #include <iostream>
2
3  // template <typename T, int Capacity> // 여러 데이터 타입을 지원하기 위한 템플릿
4  class ArrayStack
5  {
6  private:
7      int *arr;      // 배열 포인터
8      int top;       // 최상단 인덱스
9      int capacity; // 최대 용량
10
11 public:
12     ArrayStack(int capacity); // 생성자
13     ~ArrayStack();           // 소멸자
14     void push(const int &x);  // 요소 추가
15     int pop();               // 최상단 요소 제거 및 반환
16     int top() const;         // 최상단 요소 확인
17     bool empty() const;     // 비어있는지 확인
18     int size() const;       // 현재 크기 반환
19 };
```


02

스택

배열을 통한 구현

```
1  ArrayStack::ArrayStack(int capacity)
2  {
3      this->arr = new int[capacity];
4      this->top = -1;
5      this->capacity = capacity;
6  }
7
8  ArrayStack::~~ArrayStack()
9  {
10     delete[] arr;
11 }
```

```
1  void ArrayStack::push(const int &x)
2  {
3      this->arr[top + 1] = x;
4      top++;
5  }
6
7  int ArrayStack::pop()
8  {
9      if (this->empty())
10         throw std::out_of_range("stack is empty");
11     int temp = arr[top];
12     this->arr[top] = 0;
13     top -= 1;
14     return temp;
15 }
```

```
1  int ArrayStack::size() const
2  {
3      return this->top + 1;
4  }
5
6  bool ArrayStack::empty() const
7  {
8      if (top == -1)
9          return true;
10     return false;
11 }
12
13 int ArrayStack::peak() const
14 {
15     if (this->empty())
16         throw std::out_of_range("stack is empty");
17     return this->arr[top];
18 }
```

02

스택

배열을 통한 구현의 성능 분석

성능

처음 메모리를 할당할때 배열의 크기인 N 만큼 연산이 수행되고,
나머지 함수들은 상수만큼 수행된다.

공간 복잡도 - $O(N)$

시간 복잡도 - $O(1)$

한계

배열로 구현하기때문에 최대 크기가 미리 정해져 있으며, 변하지 않는다.

02

스택

링크드 리스트를 통한 구현

배열에서는 마지막 크기를 컴파일 시점에 고정하고, top을 별도로 관리해야했다.
링크드 리스트로 stack을 구현하는 경우 pointer를 활용해 top을 관리 할 수있다.

링크드 리스트로 구현하기 위해 Node class를 추가로 구현해놔야 한다.

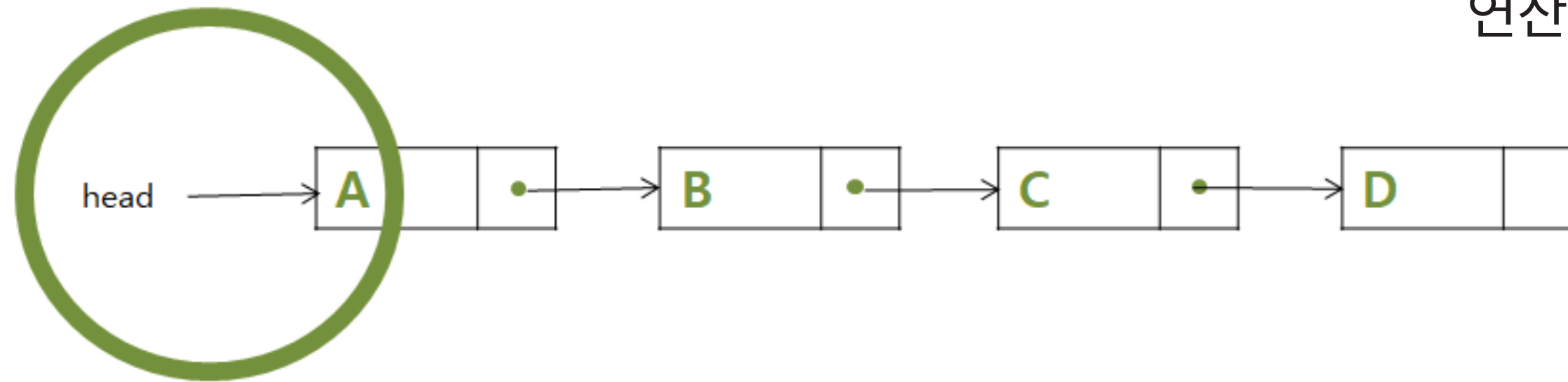
```
1  class Node
2  {
3  private:
4      int data;
5      Node *next;
6
7  public:
8      Node(int data);
9  };
10
11 class LinkedStack
12 {
13 private:
14     Node *top;
15
16 public:
17     LinkedStack();
18     ~LinkedStack();
19     void push(const int &x);
20     int pop();
21     int peak();
22     int size();
23     bool empty();
24 };
25
```

02

스택

링크드 리스트를 통한 구현

Top은 항상 Head가 된다.
연산이 Top에서만 이루어진다는것을 생각



02

스택

링크드 리스트를 통한 구현의 성능

링크드 리스트의 경우 top에서 pointer 할당 연산만 이루어지기 때문에 $O(1)$ 시간에 수행된다

배열과 달리 링크드 리스트는 메모리의 해체, 할당등의 부가 연산으로 인해 다소 느릴 수 있다.

Queue ADT

enqueue - 맨 뒤에 요소를 삽입

dequeue - 앞에서 요소를 제거함

front - 맨 앞에 요소를 반환함

size

empty

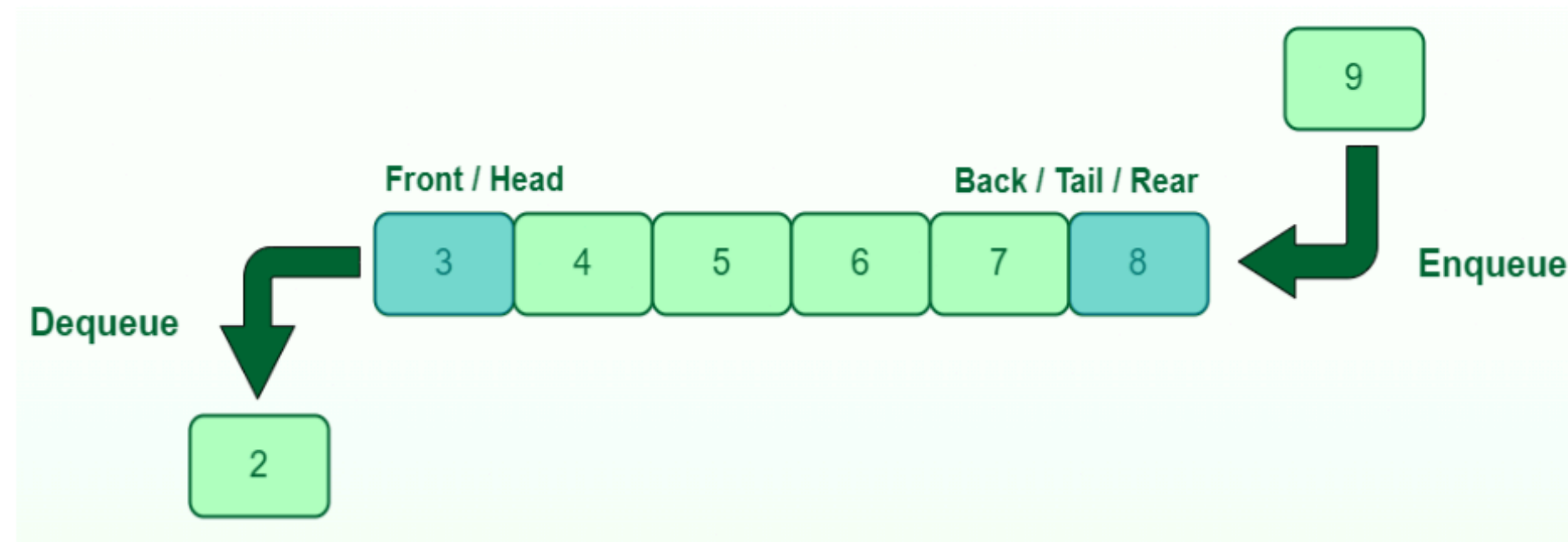
The Queue ADT

- The **Queue** ADT stores arbitrary objects
- Insertions and deletions follow the first-in first-out scheme
- Insertions are at the rear of the queue and removals are at the front of the queue
- Main queue operations:
 - **enqueue**(object): inserts an element at the end of the queue
 - **dequeue**(): removes the element at the front of the queue
- Auxiliary queue operations:
 - object **front**(): returns the element at the front without removing it
 - integer **size**(): returns the number of elements stored
 - boolean **empty**(): indicates whether no elements are stored
- Exceptions
 - Attempting the execution of dequeue or front on an empty queue throws an **QueueEmpty**

03



Queue



Queue Data Structure

Queue Interface

Queue Interface in C++

- C++ interface corresponding to our Queue ADT
- Requires the definition of exception `QueueEmpty`
- No corresponding built-in C++ class

```
template <typename E>
class Queue {
public:
    int size() const;
    bool empty() const;
    const E& front() const
        throw(QueueEmpty);
    void enqueue (const E& e);
    void dequeue()
        throw(QueueEmpty);
};
```

03

큐

배열을 통한 구현

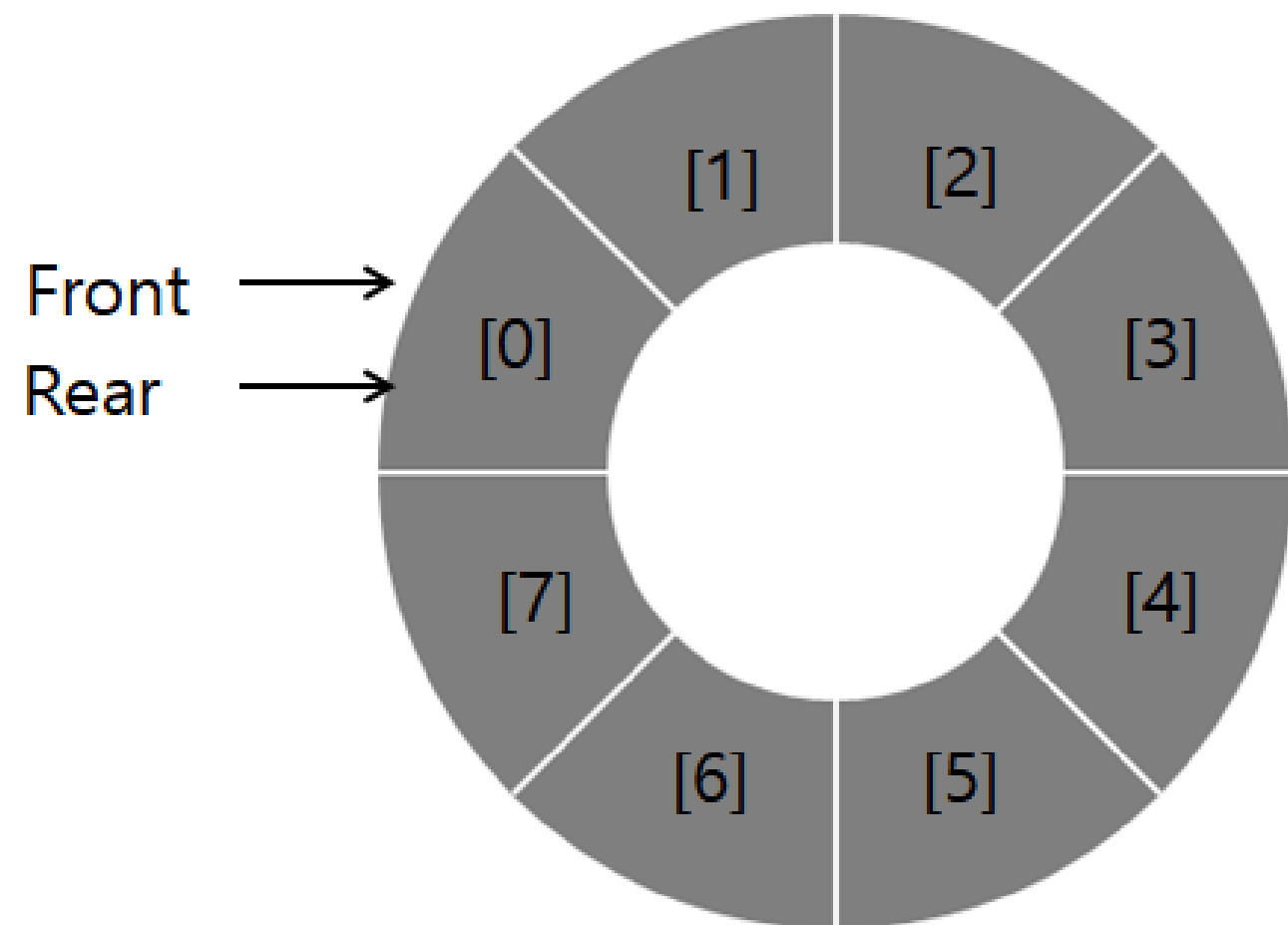
stack 에서 구현한것과 같이 배열로 구현한다.

하지만 앞으로는 데이터가 나가고, 뒤로는 데이터가 들어오기만 한다

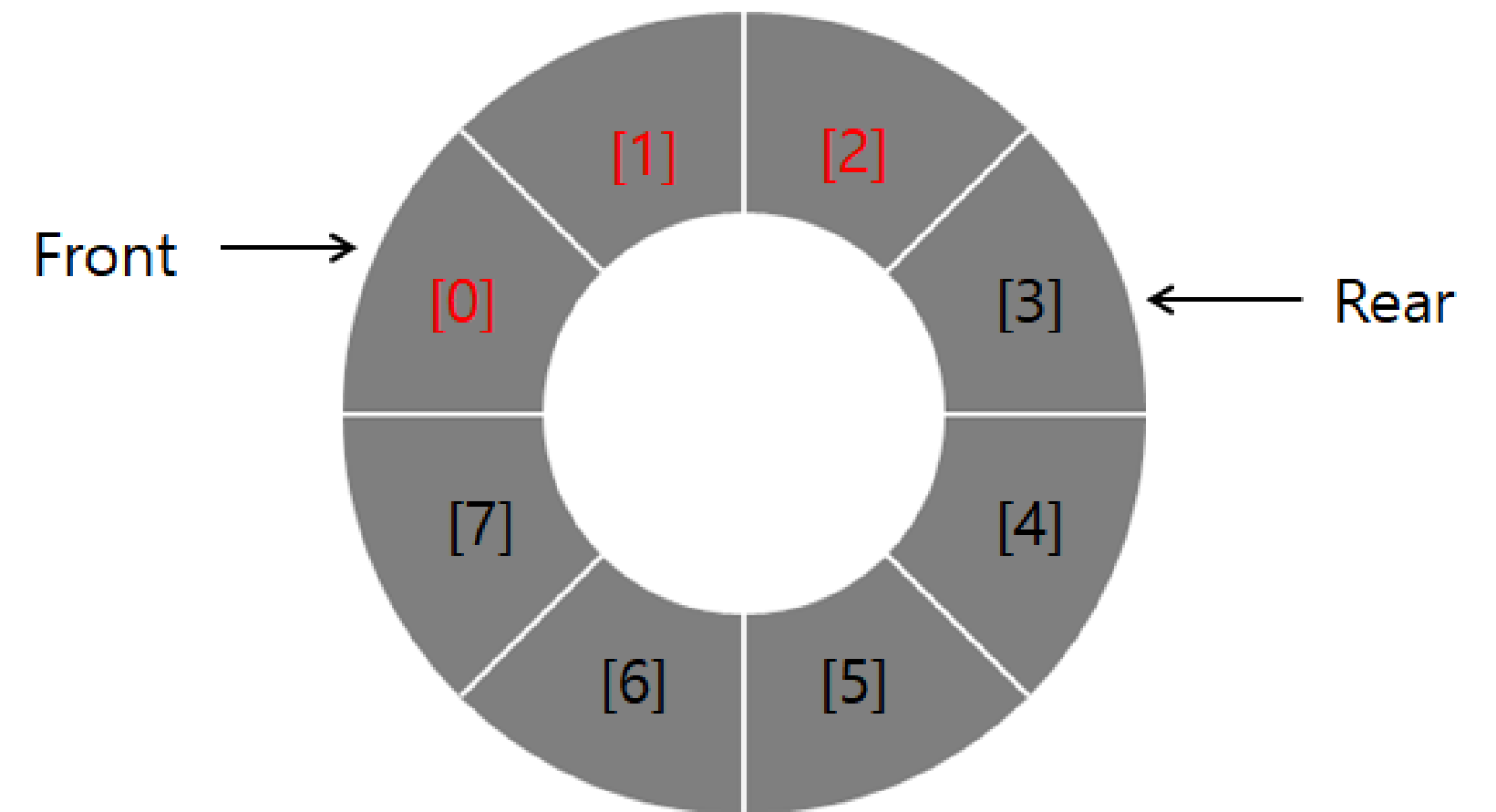
하지만 Queue에 값이 들어오는것을 생각했을때
환형배열(Circular Array) 로 구현해야한다.

```
1  #include <iostream>
2  #include <stdexcept>
3
4  template <typename T, int Capacity>
5  class ArrayQueue
6  {
7  private:
8      T *arr;          // 배열
9      int front;       // dequeue할 위치 (앞)
10     int rear;        // enqueue할 위치 (뒤)
11     int capacity;    // 최대 크기
12     int currentSize; // 현재 요소 개수
13
14 public:
15     ArrayQueue();          // 생성자
16     ~ArrayQueue();        // 소멸자
17     void enqueue(const T &x); // 후미에 추가
18     T dequeue();           // 앞에서 제거 및 반환
19     T frontElement() const; // 앞 요소 확인
20     T backElement() const;  // 뒤 요소 확인
21     bool isEmpty() const;   // 비어있음?
22     bool isFull() const;    // 꽉 참?
23     int size() const;       // 현재 크기
24 };
25
```

배열을 통한 구현



초기 상태



삽입시

03

큐

배열을 통한 구현

```
1 ArrayQueue::ArrayQueue(int capacity)
2 {
3     this->arr = new int[capacity];
4     this->front = 0;
5     this->rear = 0;
6     this->capacity = capacity;
7     this->currentSize = 0;
8 }
9
10 ArrayQueue::~~ArrayQueue()
11 {
12     delete[] arr;
13 }
14
```

```
1 void ArrayQueue::enqueue(int x)
2 {
3     if (this->full())
4     {
5         throw std::out_of_range("queue is full");
6     }
7     this->arr[this->rear] = x;
8     this->rear = (this->rear + 1) % this->capacity; // 환형
9     this->currentSize++;
10 }
11
12 int ArrayQueue::dequeue()
13 {
14     if (this->empty())
15     {
16         throw std::out_of_range("queue is empty");
17     }
18     int temp = this->arr[this->front];
19     this->front = (this->front + 1) % this->capacity; // 환형
20     this->currentSize--;
21     return temp;
22 }
23
```

```
1 int ArrayQueue::frontElement() const
2 {
3     if (this->empty())
4     {
5         throw std::out_of_range("queue is empty");
6     }
7     return this->arr[this->front];
8 }
9
10 bool ArrayQueue::empty() const
11 {
12     return this->currentSize == 0;
13 }
14
15 bool ArrayQueue::full() const
16 {
17     return this->currentSize == this->capacity;
18 }
19
20 int ArrayQueue::size() const
21 {
22     return this->currentSize;
23 }
```


03

큐

배열을 통한 구현의 성능

성능

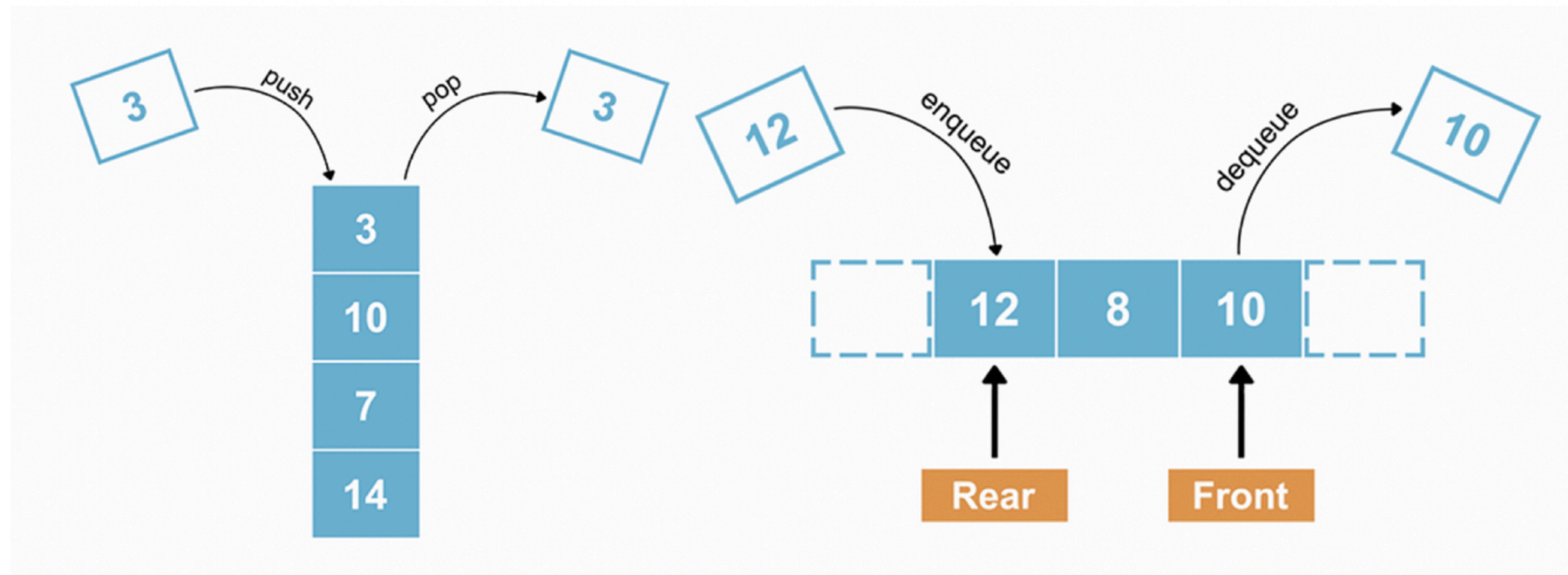
성능모든 연산 자체가 단순 산술 연산으로 상수 시간 $O(1)$ 에 수행 가능하다
다만 연산자체를 환형배열로 구현해야하기 때문에 index 관리를 해야한다.

03

큐

링크드 리스트를 통한 구현

링크드 리스트에서는 Head와 Tail이 있으므로 자연스럽게 Head-front, Tail-rear에 대응된다.



03

큐

링크드 리스트를 통한 구현

1. front와 rear이 다이렉트로 시작 노드, 마지막 노드를 가리킴
2. isEmpty() 함수가 front = NULL || n==0 을 조건으로 판별함
3. 크기에 제한이 없으므로 isFull() 함수를 사용하지 않음
4. 링크드 리스트의 실질 크기와 노드의 개수가 일치함

```
1  #include <iostream>
2  #include <stdexcept>
3
4  struct Node
5  {
6      int data;
7      Node *next;
8
9      Node(int value) : data(value), next(nullptr) {}
10 };
11
12 class LinkedQueue
13 {
14 private:
15     Node *front;    // 앞 포인터
16     Node *rear;     // 뒤 포인터
17     int currentSize; // 현재 크기
18
19 public:
20     LinkedQueue();           // 생성자
21     ~LinkedQueue();          // 소멸자
22     void enqueue(int x);     // 후미 추가
23     int dequeue();           // 앞 제거 및 반환
24     int frontElement() const; // 앞 확인
25     int backElement() const;  // 뒤 확인
26     bool empty() const;       // 비어있음?
27     int size() const;         // 현재 크기
28 };
```

03

큐

링크드 리스트를 통한 구현

```
1  LinkedQueue::LinkedQueue()
2  {
3      this->front = nullptr;
4      this->rear = nullptr;
5      this->currentSize = 0;
6  }
7
8  LinkedQueue::~~LinkedQueue()
9  {
10     while (!this->empty())
11     {
12         this->dequeue();
13     }
14 }
15
```

```
1  void LinkedQueue::enqueue(int x)
2  {
3      Node *newNode = new Node(x);
4
5      if (this->empty())
6      {
7          this->front = this->rear = newNode;
8      }
9      else
10     {
11         this->rear->next = newNode; // rear 뒤에 연결
12         this->rear = newNode;      // rear 이동
13     }
14     this->currentSize++;
15 }
16
17 int LinkedQueue::dequeue()
18 {
19     if (this->empty())
20     {
21         throw std::out_of_range("queue is empty");
22     }
23
24     Node *temp = this->front;
25     int data = temp->data;
26
27     this->front = this->front->next;
28     if (this->front == nullptr)
29     {
30         this->rear = nullptr; // 마지막 요소 삭제
31     }
32     delete temp;
33     this->currentSize--;
34
35     return data;
36 }
37
```

04

과제

스택 구현

<https://www.acmicpc.net/problem/10828>

큐 구현

<https://www.acmicpc.net/problem/10845>

백준 문제 둘 중 하나를 풀고 github에 push하기